

Koheez.ai

Technical Architecture & Risk Assessment

Version 4.0 -- July 22, 2026
Prepared for: Director of Informatics, Governance & Integration Review
Author: Engineering Team, AHF Pharmacy Division

METHODOLOGY

Every claim in this document was verified against the production codebase on July 22, 2026. File and line references are included so any statement can be independently verified. Items that are planned but not yet implemented are marked [PLANNED] or [PARTIAL]. Items that could not be independently confirmed from code are marked [NOT VERIFIED].

Contents

1. System Overview	2
2. Data Architecture	3
3. Authentication & Access Control	4
4. Integrations	5
5. Security Posture	6
6. Risk Register	7
7. Remediation Roadmap	8
8. Governance Fit	9

SNAPSHOT -- STATE AS OF JULY 22, 2026

~50+	named users, all sharing one 4-character password (AHF1) hashed at startup
72	total API routes; 28 use requireAuth middleware; remainder rely on in-handler checks
15	distinct audit event types logged; ~40 route handlers have no audit coverage
0	CSRF protection tokens issued -- no middleware installed
0	rate-limit rules on any route, including /api/auth/login
3+	third-party services receiving PHI with no confirmed BAA (OpenAI, Clerkchat, sFax)
1	shared DATABASE_URL used across development and production environments

1. System Overview

Koheez.ai is a web-based clinical decision support and pharmacy operations management platform serving AHF (AIDS Healthcare Foundation) pharmacies. Its primary functions are: (1) HIV/PrEP drug regimen assessment -- checking drug-drug interactions, renal/hepatic contraindications, and generating AI-written clinical summaries; (2) operational task management, scheduling, QA compliance tracking, and controlled substance inventory for pharmacy directors and staff; and (3) automated patient retention outreach via SMS and email. It is not an EHR, does not store full patient charts, and does not integrate with an EHR system.

Tech Stack (versions from package.json)

Runtime	Node.js (version not pinned; Replit provides ~20.x)
Language	TypeScript 5.x
Frontend	React 18.3.1 + Vite
Backend	Express.js 4.21.2
ORM	Drizzle ORM 0.39.1 + drizzle-zod 0.7.0
DB driver	@neondatabase/serverless 0.10.4 (WebSocket-based)
Session store	express-session 1.18.1 + connect-pg-simple 10.0.0
Password hashing	bcryptjs 3.0.3 (pure JS; no native bindings)
JWT validation	jose 6.2.3 (Teams SSO JWKS verification)
AI client	openai SDK 6.8.1 -> api.openai.com (NOT Azure)
Hosting	Replit (dev); Aptible-compatible (TLS terminated at proxy)
Scheduling	node-cron 4.2.1 (in-process; not a queue)

Architecture -- PHI Flow Diagram

```

Browser (React SPA)
|
| HTTPS (TLS-terminated at Replit/Aptible proxy)
|
v
Express.js API ---- OpenAI api.openai.com [PHI IN PROMPT -- NO BAA CONFIRMED]
| ---- Microsoft Graph (Outlook) [PHI: patient name, phone, msg body]
| ---- Clerkchat SMS API [PHI: patient phone + message body]
| ---- Salesforce REST API [PHI: patient activity records]
| ---- sFax API [PHI: patient PDF document]
| ---- JotForm webhook [PHI: patient service agreement PDF]
| ---- ADP Workforce Now [Employee data only -- no patient PHI]
| ---- Liverpool HIV DDI API [Drug names only -- no PHI]
| ---- NIH RxTerms API [Drug names only -- no PHI]
|
v
Neon PostgreSQL (TLS over WebSocket)
| Tables holding PHI:
| retention_patients.record (JSONB) -- initials, phone, email, address, insur. ID
| cqi_meetings.attendees (JSONB) -- staff names + signatures
| audit_logs -- staff email addresses
| client_store.value (JSONB) -- koheez_assessments: clinical notes
|
v
Browser localStorage <-- synced via /api/client-store --> PostgreSQL client_store
| koheez_assessments: clinical notes (PHI in browser)
| Cleared on logout (App.tsx)
| NOT encrypted client-side

```

PHI does NOT flow to: Liverpool HIV DDI, NIH RxTerms, ADP Workforce Now, Microsoft Entra ID JWKS endpoint. PHI DOES flow to: OpenAI (clinical notes in AI prompt), Outlook (patient outreach email), Clerkchat (patient phone + SMS body), Salesforce (patient activity), sFax (patient PDF), JotForm PDF download. BAA status for each is in Section 5.

2. Data Architecture

2.1 Schema Overview (shared/schema.ts lines 550-780)

The database uses a flat JSONB pattern: most tables store a primary key and a `record` JSONB blob containing the full typed object. Column-level queries are not possible on JSONB sub-fields without json operators. Drizzle schema was verified at shared/schema.ts.

Table	PHI Fields (if any)	Notes
retention_patients	initials, phone1, phone2, email, city, state, zip, bin, pcn, insuranceId	All PHI in JSONB `record` column. Patients can be hard-deleted (storage.ts:1274). No soft-delete. Deletion not written to audit log.
cqi_meetings	attendees[].printName, .signatureName, .userEmail	Quarterly meeting records including staff signatures. JSONB.
audit_logs	actorEmail, actorName	Staff identity only -- by design contains no patient PHI. Append-only in application code; no DELETE route.
client_store	value (JSONB: koheez_assessments blob)	koheez_assessments holds clinical assessment JSON including patient-level clinical data entered during assessment session.
qa_workbooks / qa_evidence / qa_tasks	uploadedBy (staff email)	Compliance workbook data. Evidence files stored as base64 in qa_audit_evidence.data_b64 -- not in object storage.
schedule_submissions / entries / defaults	staffName, staffId, submittedByEmail	Staff scheduling. No patient PHI.
notifications	toEmail	Staff email. No patient PHI.
pharmacy_hours	None	Store hours config only.
session	sess JSONB (user email + name)	Auto-created by connect-pg-simple. Contains authenticated user's email and name in session JSON.
adp_worker_mappings / sync_status / history	None (employee IDs)	ADP worker IDs. Employee data, not patient PHI.
wa_inspection_archives	None identified	WA state inspection checklists.
staff_time_off_balances	staffName, staffId	PTO balances. No patient PHI.

2.2 PHI Lifecycle

- Creation: Patient PHI enters via (a) manual entry in Retention Tracker UI -> POST /api/retention/patients; (b) bulk CSV/SSRS import -> POST /api/retention/import (server/routes.ts:1691); (c) clinical assessment form (drug regimen, age, labs) stored client-side in koheez_assessments localStorage key, synced to client_store table.
- Storage at rest: PostgreSQL on Neon serverless. Neon encrypts data at rest using AES-256 at the infrastructure level [NOT VERIFIED independently -- based on Neon documentation, not confirmed by code inspection]. No application-level column encryption. No pgcrypto usage found in codebase.
- Retention: No automated retention or expiry policy exists in code. Retention patients persist until manually deleted. Audit logs are append-only in the application; the PostgreSQL audit_logs table has no DELETE route and no TTL. The in-memory fallback cap (AUDIT_LOG_CAP = 10,000, storage.ts:63) applies only to the dev file-backed fallback -- the PostgreSQL table is unbounded.
- Deletion: Retention patients can be hard-deleted via DELETE /api/retention/patients/:id (requireAuth enforced; storage.ts:1274). Deletion itself is NOT logged in the audit trail -- no recordAccess call in the delete handler (routes.ts:1593-1605). No right-to-be-forgotten workflow or deletion policy exists.
- Transit: All API traffic flows over TLS terminated at the Replit/Aptible proxy. The Node process receives plain HTTP internally. Server-to-Neon uses TLS over WebSocket.

2.3 Environment Separation

As of July 22, 2026: there is one DATABASE_URL environment variable. No separate staging or production database is provisioned. Development work and production share the same Neon PostgreSQL instance. Test data, seed data written during development, and production patient records coexist in the same database. The session table is shared across environments -- a developer session on the same DATABASE_URL appears in the same session store as a production user session. This is rated HIGH severity in the risk register.

3. Authentication & Access Control

3.1 Authentication Mechanism

Authentication is session-based. On successful credential check, the server writes `req.session.user = { email, name }` and the session is persisted in the PostgreSQL ``session`` table via `connect-pg-simple`. Subsequent requests are authenticated by the presence of `req.session.user`.

Mechanism	<code>express-session 1.18.1 + connect-pg-simple 10.0.0</code>
Session secret	<code>SESSION_SECRET</code> env var (required; server throws at startup if absent). <code>server/index.ts:61-66</code> .
Cookie flags	<code>HttpOnly: true Secure: true (production only) SameSite: 'none' (required for Teams iframe) maxAge: 7 days (server/index.ts:70-78)</code>
Cookie name	Not customized -- default <code>'connect.sid'</code>
Trust proxy	<code>app.set('trust proxy', 1)</code> -- <code>server/index.ts:23</code> . Required for Secure flag behind Aptible/Replit TLS proxy.
Password hashing	<code>bcryptjs 3.0.3</code> , cost factor 10. <code>DEMO_USERS</code> hashed at module load (<code>routes.ts:170-178</code>). <code>bcrypt.compareSync</code> used at login (<code>routes.ts:460</code>).
Second factor	None implemented.
Password reset	Not implemented. No <code>/api/auth/reset-password</code> route exists.
Session fixation	<code>req.session.regenerate()</code> NOT called after login. Session ID is unchanged pre/post-authentication.

3.2 User Registry -- Known Gap (`server/routes.ts:74-169`)

User accounts are defined in a hardcoded `DEMO_USERS` array. As of this audit the array contains 53 named entries. Every entry has the source password `'AHF1'` (4 characters), hashed with `bcrypt` at startup. All users share a credential derived from the same 4-character plaintext. A compromised session or credential for any one user reveals a password that opens every other account at any role level.

There is no database-backed user table. Adding or removing a user requires a code change and redeployment. A `POST /api/auth/signup` endpoint exists (`routes.ts:499`) but is not linked to any UI and creates in-memory-only accounts that do not survive restarts. No `mustChangePassword` flag, no forced rotation, no account lockout after failed attempts.

3.3 Role Model

Role	Data Scope	Notable Permissions
CPO	All stores (national)	Audit log read + CSV export. All store data. No admin UI exists -- user management is code-only.
RPD (Regional Director)	Assigned region	Schedule approval. Regional dashboard. QA audit oversight. Cross-region access blocked via <code>getSiteAccess()</code> .
Pharmacy Director	Assigned store	CQI editing. Schedule submission. ACHC workbook. Controlled inventory. Fax log. Cross-store access blocked.
Staff Pharmacist	Assigned store	Assessment generation. Task completion. CQI signing. Inventory adjustments.
Pharmacy Technician	Assigned store	Task completion. CQI signing. Controlled inventory read-only.

3.4 Auth Enforcement Gaps (`server/routes.ts`)

Of 72 routes, 28 use the `requireAuth` Express middleware (`routes.ts:1153-1159`). The remaining routes rely on in-handler session checks via `getSiteAccess()` or role guards applied inside the handler body. The following routes have no `requireAuth` middleware and appear to have incomplete or absent auth enforcement:

Route	Line	Auth situation

POST /api/generate-note	942	No requireAuth, no in-handler session check visible
POST /api/handoff/generate	1044	No requireAuth, no in-handler session check visible
GET /api/audit-log	560	isCPO() check inside handler only (no middleware)
GET /api/audit-log/export	576	isCPO() check inside handler only (no middleware)
GET/PUT /api/scheduling/*	1850+	getSiteAccess() inside handler (functional but not middleware)
POST /api/webhooks/jotform	658	Intentionally public; X-Webhook-Secret validated

Note: getSiteAccess() reads req.session.user and returns an error response if not found -- it is functionally an auth check, but it is applied per-handler, not at the middleware layer. The /api/generate-note and /api/handoff/generate routes require independent verification of whether they enforce authentication.

No rate limiting is applied to any route, including /api/auth/login. express-rate-limit is not installed. An attacker can enumerate passwords or user accounts without triggering any server-side throttle.

4. Integrations

Service	Auth Method	PHI?	Status	Notes / Known Issues
OpenAI (api.openai.com)	API key: OPENAI_API_KEY1 or OPENAI_API_KEY env var (routes.ts:191)	YES clinical notes, drug regimens, patient context in prompt	Production	Uses api.openai.com NOT Azure OpenAI. Standard OpenAI does not include a HIPAA BAA. PHI in AI prompts without a BAA is a HIPAA violation. Azure OpenAI migration is PLANNED (Task #75) -- not complete.
Microsoft Graph (Outlook email)	Static Bearer token: OUTLOOK_ACCESS_TOKEN env var (outlookClient.ts:4)	YES patient name, phone, outreach msg body	Production	OUTLOOK_ACCESS_TOKEN is a static OAuth2 access token stored as a plain env var -- NOT a refresh token. When it expires the integration silently returns false and logs a warning. No automated rotation. Microsoft 365 / Graph is HIPAA-eligible under Microsoft BAA [NOT VERIFIED whether AHF has executed that BAA].
Clerkchat (SMS outreach)	Bearer API key: CLERKCHAT_API_KEY env var (clerkchatClient.ts:4)	YES patient phone number + SMS body	Production	No BAA confirmed. Clerkchat BAA status: MISSING / NOT VERIFIED. Patient phone numbers and outreach message content transmitted.
Salesforce (CRM activity log)	OAuth2 password grant: SF_CLIENT_ID, SF_CLIENT_SECRET, SF_USERNAME, SF_PASSWORD, SF_SECURITY_TOKEN (salesforceClient.ts:15-35)	YES patient activity records	Production	Password grant type (grant_type=password) is deprecated by Salesforce and scheduled for removal. Token cached 90 min in process memory (cleared on restart). BAA status: NOT VERIFIED -- Salesforce Health Cloud has HIPAA BAA; standard Sales Cloud may not.
JotForm (webhook + PDF)	X-Webhook-Secret header validated vs. JOTFORM_WEBHOOK_SECRET env var (routes.ts:664-672)	YES patient service agreement PDF	Production	Public endpoint (no session auth). Secret validation added in Task #137. If JOTFORM_WEBHOOK_SECRET is unset, all requests are accepted with a console warning. JotForm BAA: NOT VERIFIED.
sFax (fax to McKesson)	SFAX_USERNAME + SFAX_API_KEY env vars (faxService.ts)	YES patient PDF document	Production	sFax BAA: NOT VERIFIED. PHI in faxed content.
ADP Workforce Now	Client credentials: ADP_CLIENT_ID + ADP_CLIENT_SECRET (adpClient.ts:24-35)	NO employee data only	Production	No patient PHI transmitted. Employee name, ID, PTO data only. replit.md mentions mTLS but code uses client_id/client_secret -- mTLS status NOT VERIFIED from this code audit.
Microsoft Entra ID (Teams SSO)	JWKS validation via jose 6.2.3 (teamsAuth.ts:45-50) Requires: AAD_APP_CLIENT_ID, AAD_TENANT_ID, APP_DOMAIN	NO staff email + display name only	Production	Token validated against Microsoft public JWKS endpoint. No PHI in token. If env vars absent, SSO silently disabled; Teams users fall back to standard login.

Liverpool HIV DDI API	HTTPS REST (no API key visible in code)	NO drug names only	Production	No patient identifiers sent. Drug names only for interaction lookup.
NIH RxTerms API	Public HTTPS REST (no API key)	NO	Production	Drug name autocomplete. Public endpoint. No PHI.
Neon PostgreSQL	DATABASE_URL env var (server/db.ts:16)	YES primary PHI store	Prod + Dev (shared)	Single database for all environments. TLS on WebSocket connection. Infrastructure-level AES-256 encryption at rest [NOT VERIFIED by code -- standard Neon offering]. No application-level encryption.

5. Security Posture

5.1 Encryption in Transit

All browser-to-server traffic flows over TLS, terminated at the Replit or Aptible reverse proxy. The Node.js process itself serves plain HTTP. trust proxy is set (server/index.ts:23) so the Secure cookie flag works correctly. Server-to-Neon traffic is TLS over WebSocket. Server-to-external-API traffic is HTTPS for all integrations listed in Section 4. No non-TLS external calls were identified.

5.2 Encryption at Rest

No application-level encryption is implemented. Neon PostgreSQL provides infrastructure-level AES-256 encryption at rest per their documentation [NOT VERIFIED independently from this codebase -- based on Neon's published compliance documentation]. A direct database read using the DATABASE_URL connection string exposes all PHI in plaintext. Sensitive JSONB columns (retention_patients.record, client_store.value) are not encrypted at the column level. Evidence files uploaded to QA audit are stored as base64 text in qa_audit_evidence.data_b64 -- not in object storage and not encrypted at the application level.

5.3 Secrets Management

All secrets are stored as environment variables. No secrets manager (AWS Secrets Manager, Azure Key Vault, HashiCorp Vault) is in use. Confirmed secrets stored as plain env vars:

- SESSION_SECRET -- session signing key
- DATABASE_URL -- full Neon connection string with embedded credentials
- OPENAI_API_KEY1 / OPENAI_API_KEY -- OpenAI API key
- OUTLOOK_ACCESS_TOKEN -- static Outlook OAuth2 access token (not a refresh token; expires silently)
- SF_CLIENT_ID, SF_CLIENT_SECRET, SF_USERNAME, SF_PASSWORD, SF_SECURITY_TOKEN (5 vars for Salesforce)
- CLERKCHAT_API_KEY, CLERKCHAT_CHANNEL_ID -- Clerkchat SMS
- ADP_CLIENT_ID, ADP_CLIENT_SECRET -- ADP Workforce Now
- AAD_APP_CLIENT_ID, AAD_TENANT_ID, APP_DOMAIN -- Microsoft Entra SSO
- JOTFORM_API_KEY, JOTFORM_FORM_ID, JOTFORM_WEBHOOK_SECRET -- JotForm
- SFAX_USERNAME, SFAX_API_KEY, ICQ_FAX_NUMBER -- sFax

No rotation policy exists for any secret. The Salesforce integration stores the AHF user's Salesforce password (SF_PASSWORD) as a plain env var.

5.4 CSRF / Injection / Session Protections

CSRF tokens	NOT IMPLEMENTED. No csrf-csrf or equivalent middleware installed (not found in package.json or server/index.ts). SameSite=None cookies (required for Teams iframe embedding) combined with no CSRF tokens means state-changing requests can be forged by a malicious page sharing the browser context.
Rate limiting	NOT IMPLEMENTED. express-rate-limit not in package.json. No throttle on /api/auth/login, /api/auth/signup, or any other route.
Helmet / security headers	NOT INSTALLED. helmet package not in package.json. No Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, or Referrer-Policy headers are set.
SQL injection	Mitigated by Drizzle ORM parameterized queries. No raw SQL string interpolation found in storage.ts or routes.ts.
Input validation	Zod schemas applied to most POST/PUT request bodies. Some handlers use direct req.body access without Zod (not exhaustively audited).
XSS	React handles output escaping by default. react-markdown renders AI output -- markdown renderer config not audited for XSS mitigations.

Session fixation	NOT mitigated. express-session does not regenerate session ID on login by default. No req.session.regenerate() call found in login handler (routes.ts:437-490).
Account logout	Not implemented. Unlimited login attempts permitted.
Secure / HttpOnly cookies	Correctly configured in production (server/index.ts:70-78). SameSite=None is a known CSRF-enabling configuration when CSRF tokens are absent.

5.5 Audit Logging -- Coverage and Gaps

The HIPAA access audit log is append-only. Entries are written to both a JSON file and the PostgreSQL audit_logs table. Each entry records: at (ISO timestamp), actorEmail, actorName, role, action, resource, method, path.

Events that ARE logged (15 types):

- auth.login.success / auth.login.failure (routes.ts:462, 467)
- auth.teams-sso-login (routes.ts:526)
- assessment.generate (routes.ts:711)
- note.generate (routes.ts:951)
- client-store.write.phii -- koheez_assessments key write (routes.ts:1324)
- cqi.meeting.list / .read / .save / .sign (routes.ts:1434-1507)
- cqi.email_signers (routes.ts:1402)
- patient.read / .create / .update / .delete (routes.ts:1514-1597)
- patient.import -- bulk CSV/SSRS import (routes.ts:1782)

Actions NOT in the audit log (gaps from routes.ts line review):

- QA audit workbook reads, edits, and submissions (routes.ts:2187-2227) -- no recordAccess calls
- QA audit evidence uploads and downloads (routes.ts:2259, 2305) -- no recordAccess
- Schedule submission, approval, and override workflow -- no recordAccess in scheduling routes
- Retention patient DELETION -- handler at routes.ts:1593 exists but no audit entry is written on delete
- GET /api/client-store reads -- not logged (only writes are logged)
- Fax log reads and retry attempts -- not logged
- Any administrative action (no admin UI exists yet)

5.6 BAA Status by Vendor Receiving PHI

Vendor	BAA Status	PHI Transmitted
OpenAI (api.openai.com)	MISSING -- standard tier has no HIPAA BAA	Clinical notes, drug regimens, patient context in AI prompts
Clerkchat (SMS)	NOT VERIFIED -- presumed missing	Patient phone number, outreach message body
sFax	NOT VERIFIED	Patient PDF (service agreement)
Salesforce (standard CRM)	NOT VERIFIED -- Health Cloud has BAA; standard Sales Cloud may not	Patient contact activity records
Microsoft (Graph / Outlook)	HIPAA-eligible under Microsoft BAA -- NOT VERIFIED whether AHF has executed it	Patient name, phone, outreach email body

Neon (PostgreSQL)	NOT VERIFIED -- Neon offers HIPAA compliance tier; unclear if AHF account is on it	All PHI -- primary data store
JotForm	NOT VERIFIED	Patient service agreement PDF
ADP	N/A -- employee data only, not patient PHI	Employee scheduling data
NIH RxTerms / Liverpool DDI	N/A -- no PHI transmitted	Drug names only

6. Risk Register

Each item reflects actual code state as of July 22, 2026. 'Owner' is the party whose action is required to resolve.

#	Risk	Severity	Current Status	Owner	Target
R01	~53 users share password 'AHF1' -- compromise of one account exposes all accounts at that role level	CRITICAL	PARTIAL -- hashed at startup; plaintext 'AHF1' still in source (routes.ts:76+); per-user passwords planned in Task #138	Engineering	30 days
R02	OpenAI API receives PHI without a BAA -- HIPAA violation	CRITICAL	OPEN -- api.openai.com in production; Azure migration planned (Task #75) but not started	CPO + Legal + Eng	30 days
R03	No CSRF protection -- SameSite=None cookies + no token = forgeable state-changing requests	HIGH	OPEN -- not implemented; deferred from Task #137 due to Teams embedding complexity	Engineering	30 days
R04	Clerkchat SMS transmits patient PHI; no confirmed BAA	HIGH	OPEN	CPO + Legal	30 days
R05	Dev and prod share one DATABASE_URL -- no environment separation	HIGH	OPEN	DevOps / CPO	30 days
R06	No rate limiting on /api/auth/login or any route	HIGH	OPEN -- express-rate-limit not installed	Engineering	30 days
R07	Outlook access token is a static env var -- no refresh mechanism; expires silently	HIGH	OPEN	Engineering / IT	60 days
R08	Patient deletion not audited -- DELETE /api/retention/patients/:id writes no audit entry (routes.ts:1593)	HIGH	OPEN	Engineering	30 days
R09	Salesforce uses deprecated OAuth2 password grant; user password stored as env var	MEDIUM	OPEN	Eng / SF Admin	60 days
R10	sFax transmits patient PHI; BAA status unconfirmed	MEDIUM	OPEN	CPO + Legal	30 days
R11	QA audit workbook reads and edits not in audit log	MEDIUM	OPEN	Engineering	60 days
R12	Session ID not regenerated on login -- session fixation risk	MEDIUM	OPEN	Engineering	30 days
R13	Helmet security headers not set -- no CSP, no X-Frame-Options	MEDIUM	OPEN	Engineering	30 days

R14	No database-backed user management -- add/remove user requires code deploy	MEDIUM	OPEN -- planned Task #138	Engineering	30 days
R15	Evidence files stored as base64 in PostgreSQL, not object storage	MEDIUM	OPEN	Engineering	90 days
R16	No automated regression or security test suite	MEDIUM	OPEN -- planned Task #139	Engineering	60 days
R17	Neon and JotForm BAA status unverified	MEDIUM	OPEN	CPO + Legal	30 days
R18	ADP mTLS vs. client credential discrepancy -- replit.md describes mTLS but code uses client_id/secret	LOW	NEEDS VERIFICATION	Engineering	60 days
R19	JOTFORM_WEBHOOK_SECRET not set in prod -- accepts all requests with warning (routes.ts:672)	LOW	PARTIAL -- warning logged; env var should be provisioned	DevOps	15 days
R20	No formal security review or penetration test has been conducted	LOW	OPEN -- documented Section 8	CPO	90 days

7. Remediation Roadmap

Ordered by risk severity and implementation dependency. Items in the same band can be parallelized unless noted.

Immediate (before next review)

Ref	Action	Acceptance Criterion
R02	Migrate OpenAI calls to Azure OpenAI HIPAA-eligible endpoint OR cease sending PHI until BAA is in place	No PHI sent to api.openai.com; Azure endpoint in use with executed BAA
R04	Execute BAA with Clerkchat OR remove patient PHI from SMS content	BAA on file OR PHI removed from outreach SMS payload
R17	Confirm/execute Neon HIPAA compliance tier and JotForm BAA	Written BAA with both vendors on file
R19	Set JOTFORM_WEBHOOK_SECRET in production environment	Env var present; webhook rejects unmatched requests

30 Days

Ref	Action	Acceptance Criterion
R01	Per-user unique passwords: generate random initial creds per user; mustChangePassword flag; POST /api/auth/change-password endpoint; first-login redirect (Task #138)	No two users share plaintext password; 'AHF1' no longer valid for any account
R03	Implement csrf-csrf middleware on all state-mutating routes; add CSRF token to frontend apiRequest (Task #138). Exempt JotForm webhook and Teams SSO.	All POST/PUT/PATCH/DELETE routes require valid CSRF token
R05	Provision separate Neon project for production; update DATABASE_URL in prod secrets only	Production DB contains only production data; dev DB is separate
R06	Install express-rate-limit; apply 10 attempts/15 min window to /api/auth/login	Brute-force login returns 429 after threshold
R08	Add recordAccess('patient.delete') call in DELETE /api/retention/patients/:id handler (routes.ts:1593)	Deletion events appear in audit log
R10	Confirm or execute BAA with sFax	Written BAA on file or sFax replaced with HIPAA-covered alternative
R12	Call req.session.regenerate() after successful login (routes.ts:467)	Session ID changes on login; prior session ID invalid
R13	Install helmet; configure CSP allowing Teams iframe origins	Security headers present in all responses
R14	POST/DELETE /api/admin/users; /app/admin/users CPO UI; database-backed user table (Task #138)	Users can be added/removed without code deploy

60 Days

Ref	Action	Acceptance Criterion
R07	Replace static OUTLOOK_ACCESS_TOKEN with automated OAuth2 refresh token flow via Microsoft Graph app registration	Outlook integration renews tokens automatically; no static token in env vars
R09	Migrate Salesforce from password grant to JWT Bearer or Connected App certificate flow	No user password in env vars; token obtained via non-deprecated grant type
R11	Add recordAccess calls to QA audit workbook read, edit, submit, and evidence upload/download handlers	QA audit interactions appear in HIPAA audit log
R16	Regression test suite covering auth paths, RBAC enforcement, audit log writes, PHI lifecycle (Task #139)	CI pipeline blocks deploys on test failure

R18	Verify ADP auth method in production; document whether mTLS certs or client credentials are in use	Auth method documented and consistent with code
-----	--	---

90 Days

Ref	Action	Acceptance Criterion
R15	Move evidence file uploads from base64 PostgreSQL column to object storage (S3/Azure Blob) with server-side encryption	PHI files not stored in database; object store is HIPAA-eligible
R20	Commission external penetration test against production environment	Test report on file; critical/high findings remediated before next governance review
--	Evaluate secrets manager for all env var secrets; implement rotation policy for API keys	Secrets have defined rotation schedule; no static tokens in env vars

8. Governance Fit -- HIPAA Security Rule Technical Safeguards (45 CFR ss164.312)

The following maps the platform's current state to HIPAA Security Rule Technical Safeguard requirements. Each row states what is required, what is implemented, and what is not.

Safeguard	Current Implementation State
(a)(1) Access Control -- Unique user ID required	NOT MET: ~53 users share password 'AHF1'. Unique user IDs (email addresses) exist but unique credentials do not. Remediation in Task #138.
(a)(1) Access Control -- Automatic logoff	PARTIAL: 7-day session maxAge (server/index.ts:78). No idle-timeout or inactivity-based expiry. A session left in a browser is valid for 7 days regardless of activity.
(a)(1) Access Control -- Encryption/decryption	PARTIAL: TLS in transit confirmed. Neon infrastructure encryption at rest [not independently verified]. No application-level column encryption. Evidence files stored as base64 plaintext in DB.
(a)(2) Emergency Access Procedure	NOT VERIFIED: No documented emergency access procedure found in codebase. No break-glass account mechanism.
(b) Audit Controls	PARTIAL: HIPAA audit log with 15 event types, PostgreSQL persistence, CPO-only CSV export. Gaps: patient deletion, QA workbook access, scheduling actions not logged. No log integrity mechanism -- the audit_logs table is mutable by anyone with DATABASE_URL access.
(c) Integrity -- PHI not improperly altered/destroyed	PARTIAL: Drizzle ORM prevents SQL injection. Patients can be hard-deleted with no tombstone or recovery path. No data integrity checksums on PHI records.
(d) Person/Entity Authentication	PARTIAL: Session-based auth with bcrypt hashing (cost 10). No MFA. No certificate-based auth. Shared credential issue undermines authentication assurance level.
(e) Transmission Security	MET: TLS on all external communication paths. trust proxy correctly configured for production.

8.1 Formal Reviews This Platform Has NOT Undergone

The following is stated directly, without softening:

- No formal security review or risk analysis by a qualified information security professional has been conducted on this codebase.
- No penetration test has been performed against the production deployment.
- No formal BAA inventory has been compiled or reviewed by legal counsel. BAA status for 5 of 7 PHI-receiving vendors is unverified as of this writing.
- No data flow diagram has been formally reviewed or signed off by a HIPAA Security Officer.
- No Business Associate Agreement has been established between AHF (as covered entity) and the Koheez.ai platform itself, if the platform operates as a Business Associate.
- No Minimum Necessary standard review has been performed. The platform transmits full patient records (phone, email, insurance IDs, address) to outreach services; it has not been evaluated whether a data subset would suffice.
- No formal incident response plan references this system.
- No audit log integrity mechanism exists -- a database administrator with the DATABASE_URL connection string can alter or delete audit_logs records without detection.
- The platform has not undergone change management review for any integration added in the past 12 months (Clerkchat, sFax, ADP, Teams SSO).

8.2 What Has Been Implemented

For completeness, the following controls are in place as of this review:

- Session-based authentication with bcrypt password hashing (cost 10) -- Task #137
- SESSION_SECRET enforcement -- production startup fails without it -- Task #137
- Append-only HIPAA audit log: 15 event types, PostgreSQL persistence, CPO-only CSV export -- Task #137
- PHI (koheez_assessments) cleared from browser localStorage on logout -- Task #137

- JotForm webhook X-Webhook-Secret header validation -- Task #137
- Role-based access control: 5 tiers; site-scoped data access enforced on patient, scheduling, and CQI routes
- TLS on all external communication paths; Secure + HttpOnly cookie flags in production
- Input validation via Zod schemas on most API endpoints; Drizzle ORM prevents SQL injection
- Microsoft Entra ID JWKS token validation for Teams SSO -- eliminates Teams-based auth spoofing
- Patient import audit logging (routes.ts:1782)

This document was prepared by the Koheez.ai engineering team for internal informatics governance review. Every factual claim references a specific file and line number in the codebase as it existed on July 22, 2026 (git commit 54acee1a). Claims marked [NOT VERIFIED] reflect limitations of this code-only audit and require independent confirmation. This document does not constitute legal advice and does not substitute for a formal HIPAA risk analysis under 45 CFR ss164.308(a)(1).